```
import pandas as pd

# Load the dataset
df = pd.read_csv('/content/IIoT_Smart_Parking_Management.csv')

# Display the first 5 rows of the DataFrame
print("First 5 rows of the dataset:")
display(df.head())

# Display concise summary of the DataFrame
print("\nDataFrame Information:")
df.info()

# Display descriptive statistics of the DataFrame
print("\nDescriptive Statistics:")
display(df.describe(include='all'))
```

```
First 5 rows of the dataset:
```

	Timestamp	Parking_Spot_ID	Sensor_Reading_Proximity	Sensor_Reading_Pressure	Vehicle_Type_Weight	Vehicle_Type_Heig
0	2021-01-01 00:00:00.000000000	20	1.023651	1.541461	1831.770127	4.3925
1	2021-01-02 06:39:16.756756756	49	3.903349	1.621719	1330.815754	4.5956
2	2021-01-03 13:18:33.513513513	38	10.315709	6.292374	1255.134827	4.3137
3	2021-01-04 19:57:50.270270270	31	6.588039	1.659870	1523.442919	3.5673
4	2021-01-06 02:37:07.027027027	8	8.213969	3.278467	1758.490837	5.1458

5 rows × 28 columns

```
DataFrame Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 28 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Timestamp                  1000 non-null   object
 1   Parking_Spot_ID            1000 non-null   int64
 2   Sensor_Reading_Proximity   1000 non-null   float64
 3   Sensor_Reading_Pressure    1000 non-null   float64
 4   Vehicle_Type_Weight        1000 non-null   float64
 5   Vehicle_Type_Height        1000 non-null   float64
 6   User_Type                  1000 non-null   object
 7   Weather_Temperature        1000 non-null   float64
 8   Weather_Precipitation      1000 non-null   int64
 9   Nearby_Traffic_Level       1000 non-null   object
 10  Entry_Time                 1000 non-null   int64
 11  Exit_Time                  1000 non-null   int64
 12  Electric_Vehicle           1000 non-null   int64
 13  Reserved_Status            1000 non-null   int64
 14  Occupancy_Rate             1000 non-null   float64
 15  Payment_Amount             1000 non-null   float64
 16  Parking_Lot_Section        1000 non-null   object
 17  Payment_Status             1000 non-null   object
 18  Occupancy_Status           1000 non-null   object
 19  Vehicle_Type               1000 non-null   object
 20  Parking_Violation          1000 non-null   int64
 21  Sensor_Reading_Ultrasonic  1000 non-null   float64
 22  Parking_Duration           1000 non-null   int64
 23  Environmental_Noise_Level  1000 non-null   float64
 24  Dynamic_Pricing_Factor     1000 non-null   float64
 25  Spot_Size                  1000 non-null   object
 26  Proximity_To_Exit          1000 non-null   float64
 27  User_Parking_History       1000 non-null   float64
dtypes: float64(12), int64(8), object(8)
memory usage: 218.9+ KB

Descriptive Statistics:
```

	Timestamp	Parking_Spot_ID	Sensor_Reading_Proximity	Sensor_Reading_Pressure	Vehicle_Type_Weight	Vehicle_Type_
count	1000	1000.000000	1000.000000	1000.000000	1000.000000	1000
unique	1000	NaN	NaN	NaN	NaN	
top	2024-06-30 00:00:00.000000000	NaN	NaN	NaN	NaN	
freq	1	NaN	NaN	NaN	NaN	
mean	NaN	25.498000	5.175569	4.041883	1486.055775	4
std	NaN	14.878554	5.237664	2.816151	303.834774	0
min	NaN	1.000000	0.016117	0.051721	597.710298	1
25%	NaN	12.750000	1.379257	1.931423	1284.738158	4
50%	NaN	26.000000	3.656676	3.458518	1486.160440	4
75%	NaN	38.250000	7.145284	5.376865	1675.790880	5
max	NaN	50.000000	37.208615	27.220465	2472.927891	8

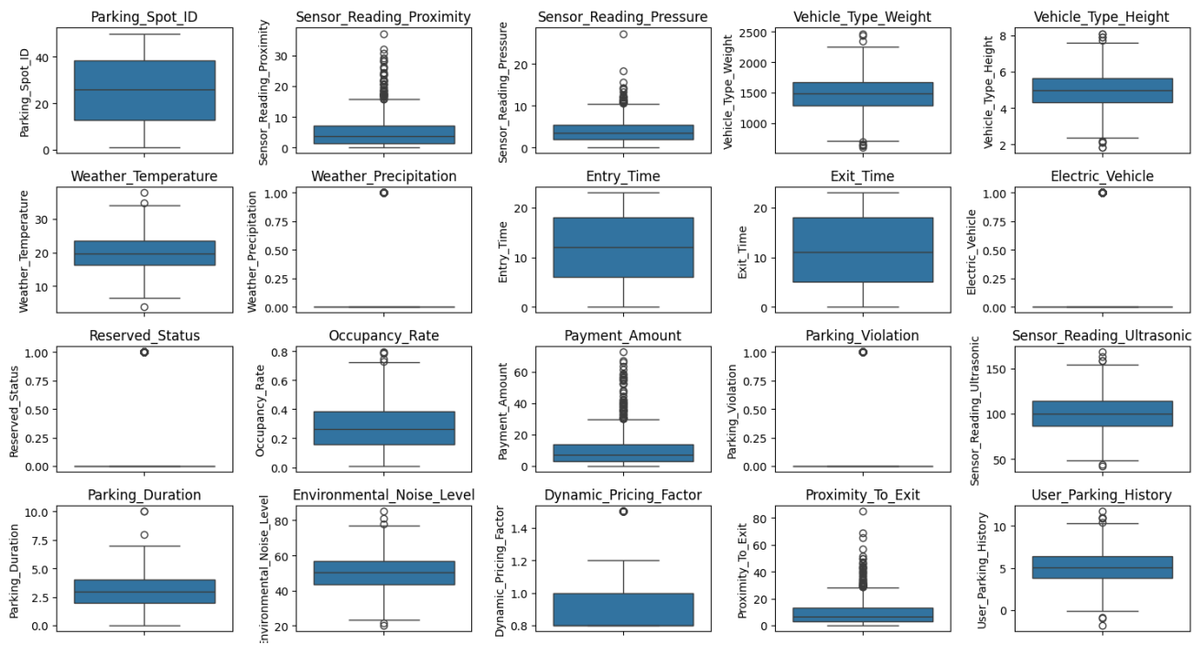
11 rows × 28 columns

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Identify numerical columns for outlier detection
numerical_cols = df.select_dtypes(include=['number']).columns

# Plot box plots for numerical columns to visualize outliers
plt.figure(figsize=(15, 10))
for i, col in enumerate(numerical_cols):
    plt.subplot(5, 5, i + 1) # Adjust subplot grid based on the number of numerical columns
    sns.boxplot(y=df[col])
    plt.title(col)
    plt.tight_layout()
plt.show()

print("\nNumerical columns in the dataset:")
print(numerical_cols)
```




```
Numerical columns in the dataset:
Index(['Parking_Spot_ID', 'Sensor_Reading_Proximity',
       'Sensor_Reading_Pressure', 'Vehicle_Type_Weight', 'Vehicle_Type_Height',
       'Weather_Temperature', 'Weather_Precipitation', 'Entry_Time',
       'Exit_Time', 'Electric_Vehicle', 'Reserved_Status', 'Occupancy_Rate',
       'Payment_Amount', 'Parking_Violation', 'Sensor_Reading_Ultrasonic',
       'Parking_Duration', 'Environmental_Noise_Level',
       'Dynamic_Pricing_Factor', 'Proximity_To_Exit', 'User_Parking_History'],
      dtype='object')
```

```
# Check for missing values
print("\nMissing values per column:")
print(df.isnull().sum())
```

```
Missing values per column:
Timestamp                    0
Parking_Spot_ID              0
Sensor_Reading_Proximity     0
Sensor_Reading_Pressure      0
Vehicle_Type_Weight          0
Vehicle_Type_Height          0
User_Type                    0
Weather_Temperature          0
Weather_Precipitation        0
Nearby_Traffic_Level         0
Entry_Time                   0
Exit_Time                    0
Electric_Vehicle             0
Reserved_Status              0
Occupancy_Rate               0
Payment_Amount               0
Parking_Lot_Section          0
Payment_Status               0
Occupancy_Status             0
Vehicle_Type                 0
Parking_Violation            0
Sensor_Reading_Ultrasonic    0
Parking_Duration             0
Environmental_Noise_Level    0
Dynamic_Pricing_Factor       0
Spot_Size                    0
Proximity_To_Exit            0
User_Parking_History         0
dtype: int64
```

Gemini

```
# Identify numerical columns
numerical_cols = df.select_dtypes(include=['number']).columns

# Handle outliers using the IQR method (capping)
for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap the outliers
    df[col] = df[col].clip(lower=lower_bound, upper=upper_bound)

print("Outliers handled using IQR capping method for numerical columns.")

# Display descriptive statistics after outlier handling
print("\nDescriptive Statistics after Outlier Handling:")
display(df.describe(include='all'))
```

```
Outliers handled using IQR capping method for numerical columns.

Descriptive Statistics after Outlier Handling:
```

	Timestamp	Parking_Spot_ID	Sensor_Reading_Proximity	Sensor_Reading_Pressure	Vehicle_Type_Weight	Vehicle_Type_
count	1000	1000.000000	1000.000000	1000.000000	1000.000000	1000
unique	1000	NaN	NaN	NaN	NaN	
top	2024-06-30 00:00:00.000000000	NaN	NaN	NaN	NaN	
freq	1	NaN	NaN	NaN	NaN	
mean	NaN	25.498000	4.898635	3.980317	1485.871293	4
std	NaN	14.878554	4.334314	2.586732	301.678839	0
min	NaN	1.000000	0.016117	0.051721	698.159074	2
25%	NaN	12.750000	1.379257	1.931423	1284.738158	4
50%	NaN	26.000000	3.656676	3.458518	1486.160440	4
75%	NaN	38.250000	7.145284	5.376865	1675.790880	5
max	NaN	50.000000	15.794324	10.545026	2262.369964	7

11 rows × 28 columns

Double-click (or enter) to edit